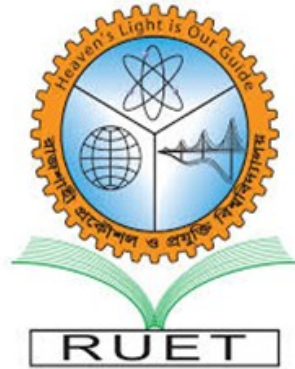

Heaven's Light is Our Guide



Rajshahi University of Engineering & Technology (RUET)

Department of Electrical and Computer Engineering

Lab report-01

Course title: Digital Techniques Sessional

Course Code: ECE 2112

Submitted Date: 26 July 2026

Submitted By

Md. Saimur Rahman Seum
Roll: 2410027

Submitted To

MST. Mazeda Noor Tasnim
Lecturer,
Dept. of ECE, RUET

Logisim Evolution:

Logisim Evolution is a free, open-source digital logic circuit simulation and design software. It is widely used by students, teachers, and engineers to design, simulate, and test digital circuits without using physical hardware.

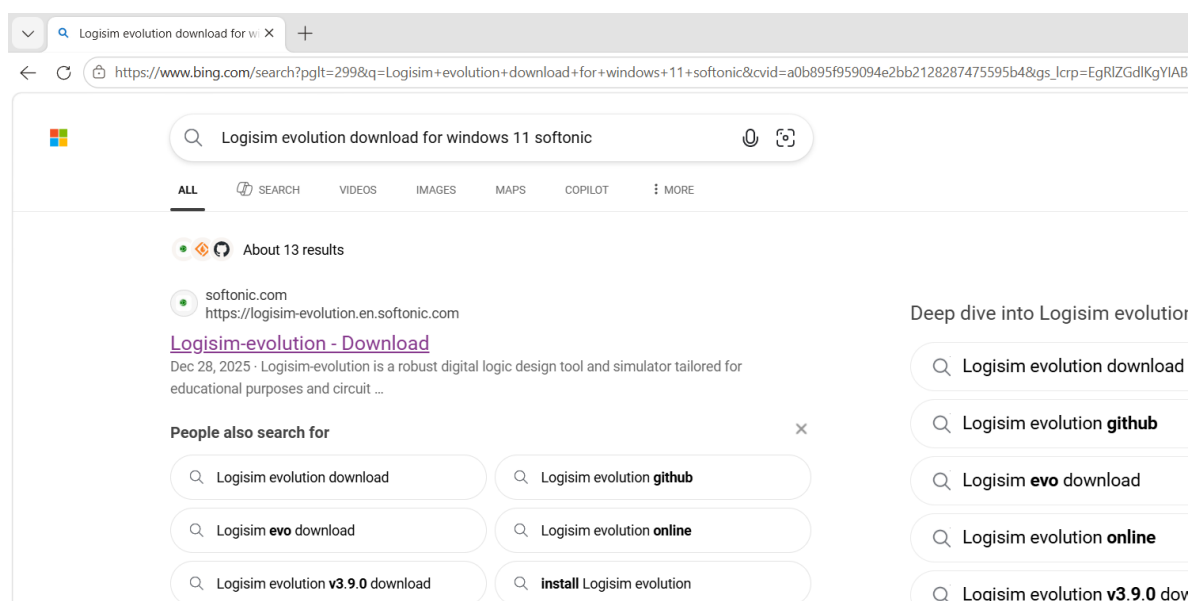
Key Features

- Design combinational and sequential logic circuits.
- Simulate logic gates, multiplexers, decoders, counters, registers, and memory.
- Build and test arithmetic circuits such as adders, subtractors, and converters.
- Support for digital displays such as 7-segment displays, LEDs, and hexadecimal displays.
- Allows hierarchical circuit design using sub-circuits.
- Provides real-time simulation and debugging of digital circuits.

Applications

- Learning digital electronics.
- Designing and testing digital logic circuits.
- Verifying circuit functionality before hardware implementation.
- Educational laboratory experiments and projects.

Installation Process:



Windows / Utilities & Tools / Logisim-Evolution / Download

Download Logisim-evolution for PC

Free ★ 4.8 (1.8k) ⬇ 9.8K
Trusted Program ↘ 4.0.0

Logisim-evolution free download. Always available from the Softonic servers

- ✓ Free & fast download
- ✓ Always available
- ✓ Tested virus-free

Free Download for PC

Alternative Logisim-evolution download from external server (availability not guaranteed)

Windows / Utilities & Tools / Logisim-Evolution

Logisim-evolution for Windows

Free ★ 4.8 (1.8k) ⬇ 9.8K
Trusted Program ↘ 4.0.0

Free Download for Windows

App specs

License: Free
Version: 4.0.0
Latest update: December 28, 2025
Platform: Windows
Other platforms (1)

Information Safety scan

Comprehensive Digital Logic Design Tool

Logisim-evolution is a robust digital logic design tool and simulator tailored for educational purposes and circuit design. This free software runs on Windows and provides users with a user-friendly interface to create and simulate digital circuits. It supports various components and allows for the construction of complex logic diagrams, making it an ideal choice for students and professionals alike.

logisim-evolution Setup

Welcome to the logisim-evolution Setup Wizard

The Setup Wizard allows you to change the way logisim-evolution features are installed on your computer or to remove it from your computer. Click Next to continue or Cancel to exit the Setup Wizard.

logisim-evolution Setup Wizard was interrupted

logisim-evolution setup was interrupted. Your system has not been modified. To install this program at a later time, please run the installation again. Click the Finish button to exit the Setup Wizard.

File Edit Project Simulate FPGA Window Help

main of Untitled · Logisim-evolu

Design Simulate

Untitled

- main
- Wiring
- Gates
- Plexers
- Arithmetic
- Memory
- Input/Output
- TTL
- TCL
- BFH mega functions
- Input/Output Extra
- System On a Chip

Properties State

Problem-01: Implementation of an OR gate using only NOR gates.

Objective:

To design and verify an OR gate using two NOR gates, and to observe how universal gates can replicate the functionality of other basic gates.

Theory:

A NOR gate is a universal gate because any logic gate can be constructed using only NOR gates. The output of a NOR gate is the complement of the OR operation.

NOR gate equation:

$$\overline{A + B}$$

To obtain an OR gate, the output of the first NOR gate is inverted using another NOR gate with its inputs connected together.

Expression:

$$Y = \overline{\overline{A + B}} = A + B$$

Thus, an OR gate can be implemented using two NOR gates.

Truth table:

Input A	Input B	NOR Output $\overline{A + B}$	Final Output (OR) $A + B$
0	0	1	0
0	1	0	1
1	0	0	1
1	1	0	1

Circuit Diagram:

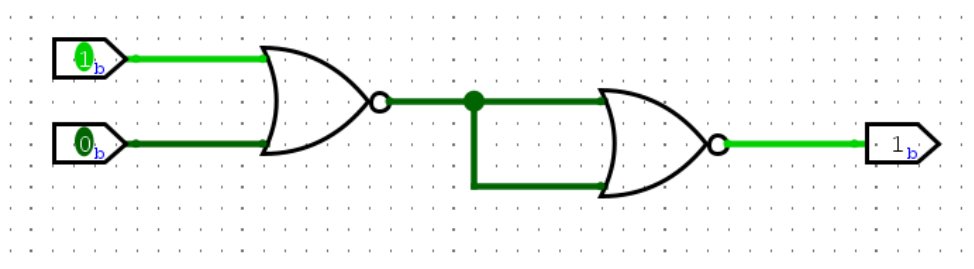


Figure 1 : OR using NOR

Discussion:

In this experiment, an OR gate was successfully implemented using two NOR gates. The first NOR gate produced the complement of the OR function, and the second NOR gate acted as an inverter to obtain the OR output. The observed output matched the expected truth table, confirming the correct operation of the circuit.

Conclusion:

The experiment verified that an OR gate can be implemented using only NOR gates. This demonstrates that the NOR gate is a universal logic gate, capable of realizing any digital logic function.

Problem-02: Implementation of an AND gate using only NOR gates.

Objective:

To implement an AND gate using only NOR gates and verify its truth table.

Theory:

A NOR gate is a universal logic gate, meaning it can be used to implement any other logic gate. To realize an AND gate using only NOR gates, the inputs are first inverted using NOR gates configured as NOT gates. These inverted inputs are then applied to another NOR gate.

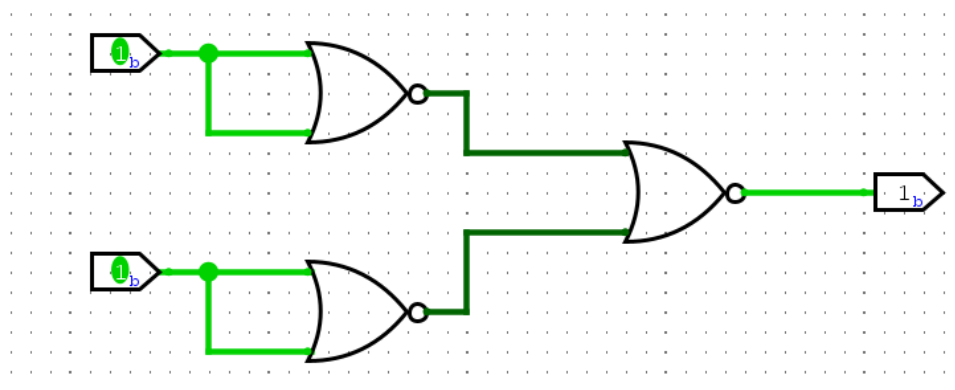
Using De Morgan's theorem:

$$A \cdot B = \overline{\overline{A} + \overline{B}}$$

Thus, an AND gate can be implemented using three NOR gates.

Truth table:

A	B	$\overline{A}$	$\overline{B}$	Final Output (AND) $A \cdot B$
0	0	1	1	0
0	1	1	0	0
1	0	0	1	0
1	1	0	0	1

Circuit Diagram:**Figure 2: AND using NOR****Discussion:**

In this experiment, an AND gate was successfully implemented using three NOR gates. The first two NOR gates acted as NOT gates to invert the inputs, while the third NOR gate combined the inverted inputs to produce the AND output. The observed outputs matched the expected truth table, confirming the correct implementation.

Conclusion:

The experiment verified that an AND gate can be implemented using only NOR gates. This demonstrates the universality of the NOR gate and its importance in digital logic circuit design.

Problem-03: Implementation of a NOT gate using only NOR gates.**Objective:**

To implement a NOT gate using only a NOR gate and verify its truth table.

Theory:

A NOR gate is a universal logic gate that can be used to implement any logic function. A NOT gate is obtained by connecting both inputs of a NOR gate together. If the common input is A , the output is:

$$Y = \overline{A + A} = \overline{A}$$

Thus, a single NOR gate can function as a NOT gate.

Truth table:

Input A	Output $\bar{A}$
0	1
1	0

Circuit Diagram:

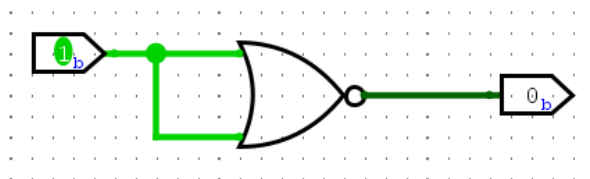


Figure 3: NOT using NOR

Discussion:

In this experiment, a NOT gate was successfully implemented using a single NOR gate by connecting both inputs together. The output was the complement of the input, and the observed results matched the expected truth table, confirming the correct operation of the circuit.

Conclusion:

The experiment verified that a NOT gate can be implemented using only a NOR gate. This demonstrates the universality of the NOR gate and its ability to realise basic digital logic functions.

Problem-04: Implementation of an OR gate using only NAND gates.

Objective:

To implement an OR gate using only NAND gates and verify its truth table.

Theory:

A NAND gate is a universal logic gate, meaning it can be used to implement any logic function. According to De Morgan's theorem:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

The inputs are first inverted using NAND gates configured as NOT gates, and then a third NAND gate combines the inverted inputs to produce the OR output. Thus, an OR gate can be implemented using three NAND gates.

Truth table:

A	B	$\overline{A}$	$\overline{B}$	Final Output (OR) A+B
0	0	1	1	0
0	1	1	0	1
1	0	0	1	1
1	1	0	0	1

Circuit Diagram:

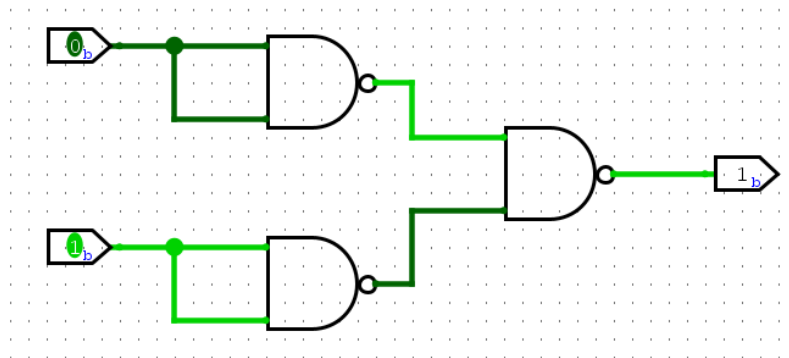


Figure 4: OR using NAND

Discussion:

In this experiment, an OR gate was successfully implemented using three NAND gates. The first two NAND gates acted as NOT gates by inverting the inputs, while the third NAND gate combined the inverted inputs to produce the OR function. The observed outputs matched the expected truth table, confirming the correct operation of the circuit.

Conclusion:

The experiment verified that an OR gate can be implemented using only NAND gates. This demonstrates that the NAND gate is a universal logic gate, capable of implementing any digital logic function.

Problem-05: Implementation of an AND gate using only NAND gates.

Objective:

To implement an AND gate using only NAND gates and verify its truth table.

Theory:

A NAND gate is a universal logic gate, meaning it can be used to implement any logic function. A NAND gate produces the complement of the AND operation:

$$Y = \overline{A \cdot B}$$

To obtain an AND gate, the output of the first NAND gate is inverted using a second NAND gate with its two inputs connected together.

Expression:

$$Y = \overline{\overline{A \cdot B}} = A \cdot B$$

Thus, an AND gate can be implemented using two NAND gates.

Truth table:

A	B	NAND Output $\overline{A \cdot B}$	Final Output (AND) $A \cdot B$
0	0	1	0
0	1	1	0
1	0	1	0
1	1	0	1

Circuit Diagram:

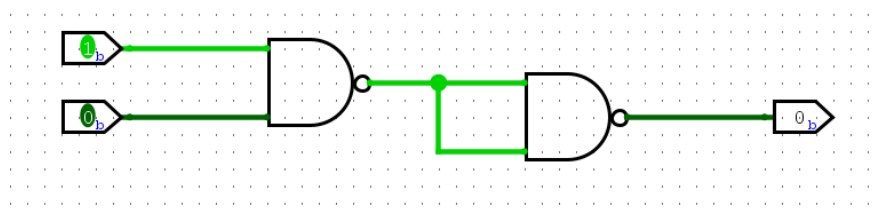


Figure 5: AND using NAND

Discussion:

In this experiment, an AND gate was successfully implemented using two NAND gates. The first NAND gate generated the complement of the AND function, and the second NAND gate acted as an inverter by connecting its inputs together. The observed outputs matched the expected truth table, confirming the correct implementation of the AND gate.

Conclusion:

The experiment verified that an AND gate can be implemented using only NAND gates. This demonstrates the universality of the NAND gate and its ability to realise any digital logic function.

Problem-06: Implementation of a NOT gate using only NAND gates.**Objective:**

To implement a NOT gate using only a NAND gate and verify its truth table.

Theory:

A **NAND gate** is a universal logic gate that can be used to implement any logic function. A NOT gate is obtained by connecting both inputs of a NAND gate together. If the common input is A , the output is:

$$Y = \overline{A \cdot A} = \overline{A}$$

Thus, a single NAND gate can function as a NOT gate.

Truth table:

Input A	Output $\overline{A}$
0	1
1	0

Circuit Diagram:

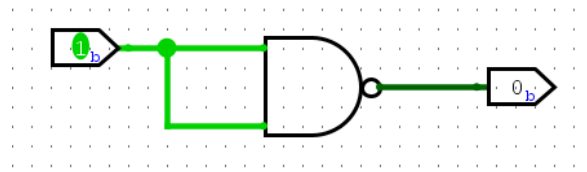


Figure 6: NOT using NAND

Discussion:

In this experiment, a NOT gate was successfully implemented using a single NAND gate by connecting both of its inputs together. The gate produced the complement of the input signal, and the observed outputs matched the expected truth table, confirming the correct operation of the circuit.

Conclusion:

The experiment verified that a NOT gate can be implemented using only a NAND gate. This demonstrates the universality of the NAND gate and its capability to realise basic as well as complex digital logic functions.

Problem-07: Implementation of a full adder circuit and testing it against its truth table.

Objective:

To implement a full adder circuit and verify its operation using the truth table.

Theory:

A full adder is a combinational logic circuit that adds three 1-bit binary inputs: two operands (A and B) and a carry-in (C_{in}). It produces two outputs: Sum (S) and Carry-out (C_{out}).

The Boolean expressions are:

- Sum: $S = A \oplus B \oplus C_{in}$
- Carry: $C_{out} = AB + AC_{in} + BC_{in}$

A full adder can be implemented using two half adders and one OR gate. It is widely used in binary addition circuits and arithmetic logic units (ALUs).

Truth table:

A	B	Cin	Sum(S)	Carry(Cout)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

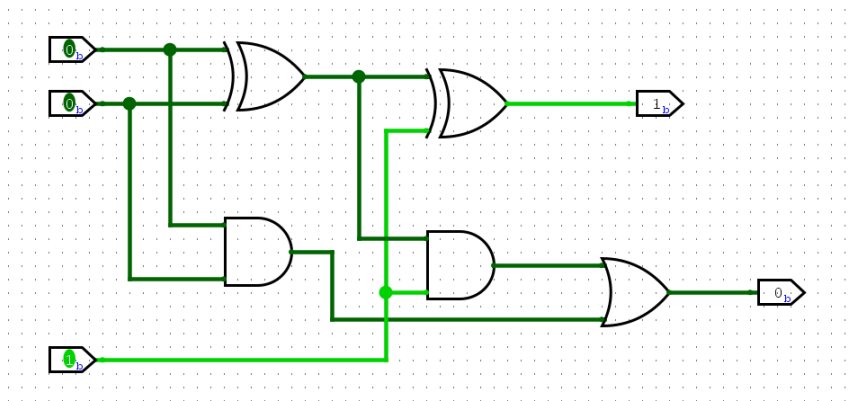
Circuit Diagram:

Figure 7: Full adder circuit

Discussion:

In this experiment, a full adder circuit was constructed and tested for all possible input combinations of A, B, and Cin. The outputs Sum and Carry-out were observed and compared with the theoretical truth table. The experimental results matched the expected outputs, confirming the correct operation of the full adder circuit.

Conclusion:

The experiment successfully implemented a full adder circuit and verified its functionality using the truth table. The results confirmed that the circuit correctly performs binary addition of three input bits and generates the appropriate Sum and Carry-out outputs.

Problem-08: Implementation of a binary to BCD converter for a 4-bit binary input.

Objective:

To implement a 4-bit Binary to BCD converter and verify its operation using the truth table.

Theory:

A Binary to BCD (Binary-Coded Decimal) converter converts a binary number into its equivalent BCD representation. In this experiment, an 8-bit binary number (0–255) is applied to the converter. The output is displayed on three BCD displays representing the hundreds (100), tens (10), and ones (1) digits. Each decimal digit is encoded separately as a 4-bit BCD code, making the output suitable for digital displays and numerical processing.

Truth table:

Binary input	Decimal output	Hundreds	Tens	Ones
00000000	0	0	0	0
00000001	1	0	0	1
00001010	10	0	1	0
00011111	31	0	3	1
01100100	100	1	0	0
11101011	235	2	3	5
11111111	255	2	5	5

Circuit Diagram:

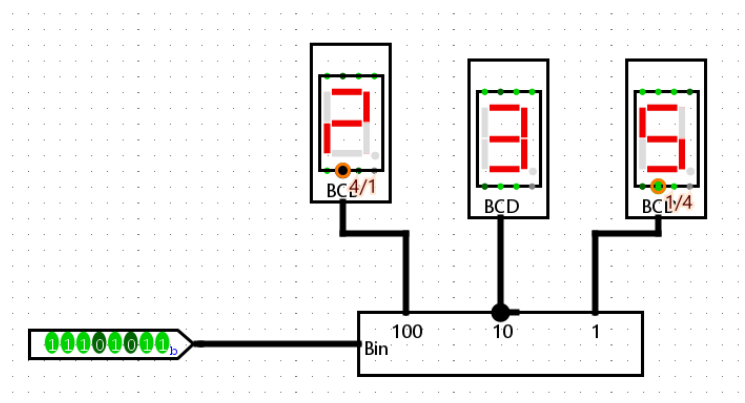


Figure 8: Binary to BCD

Discussion:

The Binary to BCD converter was implemented and tested using different 8-bit binary inputs. The converter successfully transformed each binary number into its corresponding decimal digits, which were displayed on the three BCD displays representing the hundreds, tens, and ones places. For example, when the binary input represented 235, the displays showed 2, 3, and 5, confirming correct conversion. The observed outputs agreed with the expected decimal values.

Conclusion:

The Binary to BCD converter was successfully implemented and tested. The circuit correctly converted 8-bit binary numbers (0–255) into their equivalent BCD outputs and displayed the decimal digits on the three BCD displays. The experimental results matched the expected outputs, confirming the proper operation of the converter.

References:

1. M. M. Mano, Digital Design, 5th ed. Upper Saddle River, N.J.: Pearson, 2013.
2. R. J. Tocci, N. S. Widmer, and G. L. Moss, Digital Systems: Principles and Applications, 11th ed. Upper Saddle River, N.J.: Pearson, 2011.